ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

Ngày nhận hồ sơ	
(Do CQ quản lý ghi)	

THUYẾT MINH
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN 2026

A. THÔNG TIN CHUNG

A1. Tên đề tài

- Tên tiếng Việt (IN HOA): **ĐÁNH GIÁ THỰC NGHIỆM CHIẾN LƯỢC PHÂN ĐOẠN MÃ NGUỒN TRONG MÔ HÌNH RAG HỖ TRỢ ĐỐI SÁNH NĂNG LỰC ỨNG VIÊN**
- Tên tiếng Anh (IN HOA): **AN EMPIRICAL EVALUATION OF SOURCE CODE CHUNKING STRATEGIES IN RAG MODELS FOR CANDIDATE SKILL MATCHING**

A2. Thời gian thực hiện

6 tháng (kể từ khi được duyệt).

A3. Tổng kinh phí

Tổng kinh phí: …**1**.. triệu đồng, gồm

- Kinh phí từ Trường Đại học Công nghệ Thông tin: ..**1**.. triệu đồng

A4. Chủ nhiệm

Họ và tên: **Phạm Viết Quang Huy**

Ngày, tháng, năm sinh: 27/03/2006 Giới tính (Nam/Nữ): Nam

Số CCCD: 070206005692; Ngày cấp:24/06/2026; Nơi cấp: Bộ Công an

Mã số sinh viên: **24520692**

Số điện thoại liên lạc: **0949809372**

Đơn vị (Khoa): **Công nghệ Phần mềm**

Số tài khoản: 3427032006 Ngân hàng: MB Bank

A5. Thành viên đề tài (kể cả CNĐT)

TT	Họ tên	MSSV	Khoa
1	Phạm Viết Quang Huy	24520692	Công nghệ Phần mềm
2	Nguyễn Ngọc Thu An	24520063	Công nghệ Phần mềm

B. MÔ TẢ NGHIÊN CỨU

B1. Giới thiệu về đề tài

Hiện nay hầu hết ứng viên đã đính kèm liên kết GitHub trong CV và nhà tuyển dụng có thể tự mở để xem. Tuy nhiên cách làm thủ công này bộc lộ nhiều giới hạn rõ rệt khi số lượng ứng viên lớn, đặc biệt ở các đợt tuyển thực tập sinh IT thường nhận hàng trăm hồ sơ cho một vị trí:

- **Không thể xử lý ở quy mô lớn:** HR không thể mở và đọc code của hàng trăm ứng viên, trên thực tế chỉ xem lướt README hoặc số star của repo, không đủ để đánh giá năng lực thực sự.
- **Thiếu liên kết trực tiếp với yêu cầu công việc:** mở GitHub chỉ thấy danh sách repo, không biết repo nào chứng minh đúng từng yêu cầu cụ thể trong JD.
- **Không có cơ chế xếp hạng hay so sánh:** nhiều ứng viên được xem độc lập, không theo cùng một bộ tiêu chí thống nhất, tốn thời gian và thiếu nhất quán.
- **Khó phát hiện dự án fork/sao chép:** ứng viên có thể để nguyên project của người khác trong profile mà HR không nhận ra khi xem lướt.
- **Sinh viên không tự định hướng được việc học:** xem GitHub của chính mình cũng không tự biết đang thiếu kỹ năng gì so với một vị trí cụ thể.

Từ thực tế đó, đề tài đề xuất xây dựng nền tảng tuyển dụng theo mô hình **Evidence-based Candidate Profile** – hồ sơ ứng viên được minh chứng bằng dữ liệu dự án GitHub thực tế thay vì mô tả chủ quan trong CV, kết hợp Mô hình ngôn ngữ lớn (LLM) và kiến trúc RAG (Retrieval-Augmented Generation) để:

- Tự động bóc tách yêu cầu kỹ năng từ JD
- Đối chiếu ngữ nghĩa với minh chứng của ứng viên.
- Xếp hạng mức độ phù hợp có thể giải thích được.
- Tự động lọc bỏ dự án fork.
- Tạo báo cáo Skill Gap giúp sinh viên biết chính xác mình cần bổ sung kỹ năng gì.

Khoảng trống nghiên cứu

Qua khảo sát các nghiên cứu liên quan, nhóm chưa tìm thấy nghiên cứu thực nghiệm tập trung trực tiếp vào việc đánh giá ảnh hưởng của chiến lược phân đoạn mã nguồn – đặc biệt là AST-based chunking so với fixed-size chunking – đối với chất lượng truy xuất bằng chứng và xếp hạng trong bài toán JD–GitHub skill matching. Trên cơ sở đó, đề tài xác định hai hướng nghiên cứu trọng tâm:

1. Xây dựng hệ thống hỗ trợ đối sánh JD và hồ sơ GitHub dựa trên bằng chứng năng lực thực tế (Evidence-based Matching), thay vì đối chiếu từ khóa hay dựa vào mô tả chủ quan trong CV.
2. Nghiên cứu thực nghiệm ảnh hưởng của chiến lược phân đoạn (chunking) mã nguồn – so sánh giữa phân đoạn theo cấu trúc ngữ nghĩa (AST-based chunking) và phân đoạn văn bản cơ học truyền thống (fixed-size chunking) – đến hiệu quả truy xuất bằng chứng và chất lượng đối sánh kỹ năng.

Bên cạnh đó, đề tài còn triển khai một số giải pháp kỹ thuật bổ trợ (xác thực đóng góp, lọc nhiễu dữ liệu, phát hiện tài khoản bot...) nhằm đảm bảo chất lượng dữ liệu đầu vào, đóng vai trò kỹ thuật công trình chứ không phải trọng tâm đóng góp khoa học của đề tài.

Câu hỏi nghiên cứu (Research Questions)

- **RQ1:** AST-based chunking có cải thiện chất lượng truy xuất bằng chứng kỹ năng từ repository so với fixed-size text chunking hay không?
- **RQ2:** Chất lượng truy xuất khác nhau giữa hai chiến lược chunking có dẫn đến sự khác biệt về chất lượng xếp hạng ứng viên hay không?
- **RQ3:** AST-based chunking đánh đổi bao nhiêu chi phí token và latency để đạt được cải thiện về retrieval và ranking?

Phạm vi đề tài: Đóng góp khoa học trọng tâm của đề tài là thực nghiệm so sánh định lượng giữa AST-based chunking và fixed-size chunking (RQ1–RQ3). Hệ thống web hai phân hệ được xây dựng ở mức MVP (Minimum Viable Product) nhằm minh họa luồng nghiệp vụ và phục vụ thu thập dữ liệu thực nghiệm, không hướng tới hoàn thiện sản phẩm thương mại. Các module bổ trợ (phát hiện sao chép mã nguồn, phát hiện bot, đánh giá độ trưởng thành repository) được trình bày ở mục Hướng phát triển, không thuộc phạm vi cam kết bắt buộc của đề tài trong 6 tháng.

B2. Mục tiêu, nội dung, kế hoạch nghiên cứu

B2.1 Mục tiêu

Mục tiêu nghiên cứu

- Xây dựng hệ thống hỗ trợ đối sánh Job Description và hồ sơ GitHub dựa trên bằng chứng năng lực (Evidence-based JD–GitHub Matching), sử dụng LLM để bóc tách yêu cầu kỹ năng từ JD và kiến trúc RAG để đối sánh ngữ nghĩa với minh chứng của ứng viên, cho phép xếp hạng có thể giải thích được.
- Nghiên cứu thực nghiệm ảnh hưởng của chiến lược phân đoạn mã nguồn (chunking strategy) đến hiệu quả truy xuất bằng chứng và chất lượng đối sánh kỹ năng, thông qua so

sánh định lượng giữa AST-based chunking (dùng tree-sitter) và phân đoạn văn bản cơ học truyền thống.

- Xây dựng bộ tiêu chí đánh giá định lượng tương ứng với ba câu hỏi nghiên cứu: Precision@k/Recall@k/F1@k trả lời RQ1, NDCG/MRR trả lời RQ2, và chi phí tính toán (token, độ trễ) trả lời RQ3, kèm kiểm định ý nghĩa thống kê giữa hai chiến lược phân đoạn.

Mục tiêu kỹ thuật hỗ trợ

- Xây dựng module phân tích khoảng trống kỹ năng (Skill Gap) giúp sinh viên biết chính xác kỹ năng còn thiếu so với vị trí mong muốn, kèm gợi ý công nghệ cần bổ sung.
- Đánh giá tính khả thi của việc dùng dữ liệu GitHub (README, cấu hình dự án, lịch sử commit) làm minh chứng năng lực ứng viên thay thế/bổ sung cho CV truyền thống.

B2.2 Nội dung và phương pháp nghiên cứu

Nội dung 1: Xây dựng hệ thống đối sánh JD – hồ sơ GitHub dựa trên bằng chứng năng lực

Nội dung: Xây dựng hệ thống đối sánh JD – hồ sơ GitHub dựa trên bằng chứng năng lực (Evidence-based JD–GitHub Matching), gồm hai phân hệ:

- **(a) Phân hệ Doanh nghiệp**: đăng tải và bóc tách JD bằng LLM; dashboard quản lý ứng viên; xếp hạng ứng viên kèm minh chứng cụ thể.
- **(b) Phân hệ Sinh viên**: khởi tạo hồ sơ tự động qua GitHub OAuth; tìm kiếm vị trí thực tập theo kỹ năng/lĩnh vực; báo cáo Skill Gap.

Phương pháp thực hiện:

- **Bóc tách JD:** dùng LLM để bóc tách yêu cầu kỹ năng từ JD thành cấu trúc chuẩn hóa.
- **Thu thập & đối chiếu dữ liệu:** áp dụng kiến trúc RAG [1] để thu thập dữ liệu GitHub, biểu diễn dưới dạng vector và đối chiếu ngữ nghĩa với yêu cầu kỹ năng đã bóc tách.
- **Biểu diễn năng lực có thể giải thích:** kế thừa nguyên lý trích xuất chuyên môn từ cú pháp mã nguồn và khả năng giải thích truy vết trực tiếp tới commit/đoạn mã minh chứng (commit-level traceability) [5] để xây dựng hồ sơ năng lực có thể giải thích được (explainable).
- **Cơ chế xếp hạng:** tham khảo hướng đối sánh JD – hồ sơ ứng viên dựa trên minh chứng GitHub trong tuyển dụng kỹ thuật [4] để thiết kế cơ chế xếp hạng ứng viên.

Dự kiến kết quả: Bản MVP cho phép doanh nghiệp đăng JD và nhận danh sách ứng viên đã xếp hạng có minh chứng cho từng yêu cầu; sinh viên có hồ sơ năng lực tự động và báo cáo Skill Gap. Giao diện tập trung vào luồng nghiệp vụ chính, không đầu tư sâu vào hoàn thiện UI/UX.

Nội dung 2: Nghiên cứu thực nghiệm ảnh hưởng của chiến lược phân đoạn mã nguồn

Nội dung: Nghiên cứu thực nghiệm ảnh hưởng của chiến lược phân đoạn (chunking) mã nguồn đến hiệu quả truy xuất bằng chứng và chất lượng đối sánh kỹ năng – đây là đóng góp khoa học cốt lõi của đề tài.

Phương pháp thực hiện:

Kiến trúc Phân bổ Lũy tiến (Progressive Disclosure) - 3 cấp:

Để giải quyết giới hạn context window của LLM và giảm chi phí gọi API, dữ liệu repository được xử lý theo 3 cấp tăng dần độ chi tiết:

- **Cấp 1 – Metadata**: đọc nhanh các tệp cấu hình (package.json, pom.xml, .csproj) để nắm bắt Tech Stack;
- **Cấp 2 – README:** trích xuất README.md để hiểu luồng nghiệp vụ dự án;
- **Cấp 3 – Mã nguồn chi tiết:** chỉ kích hoạt khi JD yêu cầu khắt khe một kỹ năng cụ thể. Đây là nơi triển khai và so sánh định lượng hai chiến lược chunking.

Hai chiến lược chunking được so sánh (trọng tâm thực nghiệm ở Cấp 3):

- **AST-based chunking:** Sử dụng Tree-sitter để phân đoạn theo từng định nghĩa hàm/phương thức (function/method definition) — một biến thể được tinh gọn từ thuật toán gộp/tách theo ngưỡng kích thước trong cAST [2] nhằm phù hợp với phạm vi thực hiện 6 tháng. Nhằm tránh phân mảnh ngữ cảnh khi tạo vector embedding và truy xuất, mỗi chunk mã nguồn được tự động gắn kèm siêu dữ liệu ngữ cảnh (context header) gồm đường dẫn tệp tin, tên lớp (class/interface) và các khai báo thư viện liên quan (import/require statements).
- **Fixed-size chunking:** chia theo số token cố định (ví dụ 500 token/chunk, overlap 50 token), tương tự RecursiveCharacterTextSplitter của LangChain.

Giới hạn phạm vi & xử lý lỗi:

Do phân tích AST bằng Tree-sitter dễ thất bại khi mã nguồn chứa lỗi cú pháp hoặc thiếu file phụ thuộc, đồng thời việc hỗ trợ cùng lúc nhiều ngôn ngữ lập trình làm tăng đáng kể độ phức tạp cài đặt:

- Phạm vi AST-based chunking giới hạn ở hai ngôn ngữ phổ biến nhất trong dataset: **Java** và **JavaScript**/**TypeScript**.
- Cơ chế fallback: nếu Tree-sitter không parse được AST do lỗi cú pháp hoặc thiếu file phụ thuộc, pipeline tự động chuyển sang Line-based/Indent-based hoặc Recursive Character Splitter đảm bảo mọi mẫu dữ liệu đều được xử lý trọn vẹn, không làm gián đoạn quy trình.

Hạ tầng thực nghiệm cố định (đảm bảo tính tái lập):

- Embedding Model: **bge-m3**
- Vector Database: **pgvector**
- Evaluator LLM (LLM-as-a-Judge, sinh giải thích): **Gemini 1.5 Flash**

Chiến lược tối ưu chi phí:

Để kiểm soát chi phí trong điều kiện ngân sách hạn chế và tránh giới hạn tốc độ gọi (rate-limit) khi lặp lại nhiều vòng thử nghiệm, nhóm sử dụng mô hình mã nguồn mở chạy local qua Ollama (Llama-3/Qwen2.5) kết hợp Local Embedding (BGE-M3) trên máy trạm cá nhân có GPU hoặc server phòng thí nghiệm (nếu có) trong giai đoạn phát triển và chạy ablation study diện rộng. API thương mại (Gemini 1.5 Flash) chỉ được dùng ở giai đoạn benchmark chính thức cuối cùng.

Dự kiến kết quả: Bằng chứng thực nghiệm định lượng cho thấy chiến lược phân đoạn nào mang lại hiệu quả truy xuất và đối sánh kỹ năng tốt hơn, đồng thời tối ưu chi phí token và độ trễ xử lý.

Nội dung 3: Xây dựng bộ tiêu chí và thực hiện đánh giá định lượng (Benchmark & Evaluation)

Nội dung: Xây dựng bộ tiêu chí và thực hiện đánh giá định lượng nhằm trả lời trực tiếp ba câu hỏi nghiên cứu RQ1–RQ3.

Phương pháp thực hiện:

Dataset:

- **50 JD** thu thập từ tin tuyển dụng thực tập sinh/fresher IT thực tế (ITviec, TopCV), phủ các mảng Backend/Frontend/Mobile.
- **100 repository** GitHub công khai.

Gán nhãn Ground Truth:

- Thang 3 mức theo Annotation Guideline: **0 -** không liên quan, **1 -** liên quan gián tiếp, **2 -** minh chứng trực tiếp.
- Thực hiện bởi 2 người đánh giá độc lập, tính hệ số đồng thuận **Weighted Cohen's Kappa** (dùng bản có trọng số do thang nhãn 0/1/2 là thang thứ tự, không phải danh mục) (**mục tiêu ≥ 0.7**).
- Áp dụng **Hybrid Evaluation (LLM-as-a-Judge)** để giảm khối lượng gán nhãn thủ công: Gemini 1.5 Flash gán nhãn và chấm điểm sơ bộ toàn bộ dataset, con người thẩm định lại 20–30% mẫu ngẫu nhiên để kiểm soát độ chính xác.
- Nhãn Relevance cho chỉ số NDCG được quy đổi cụ thể: *Relevance(ứng viên) = (số kỹ năng trong JD có minh chứng code) / (tổng số kỹ năng JD yêu cầu).*

Bộ chỉ số đánh giá:

- **Chất lượng truy xuất & xếp hạng:** Precision@5, Recall@5, F1@5, NDCG@5, MRR (cố định k=5 để đối chiếu trực tiếp với kết quả đã công bố của cAST [2]).
- **Chi phí vận hành:** token, latency.
- **Chất lượng RAG chuyên dụng (Ragas – RAG Triad [3]):** Faithfulness (mức độ câu trả lời của LLM bám sát code chunk truy xuất được, kiểm soát ảo giác), Context Precision (tỷ lệ ngữ cảnh truy xuất thực sự liên quan), và Context Recall (tỷ lệ ngữ cảnh liên quan thực sự tồn tại mà pipeline truy xuất được, đo khả năng "không bỏ sót" của mỗi chiến lược chunking).
- **Thống kê mô tả corpus:** số lượng chunk trung bình và kích thước chunk trung bình mỗi repository theo từng chiến lược, nhằm minh bạch hóa khác biệt phân bố dữ liệu giữa hai chiến lược trước khi so sánh trực tiếp Precision/Recall — theo đúng lưu ý về khác biệt phân bố corpus mà cAST [2] đã chỉ ra.

Kiểm định & Ablation Study:

- Kiểm định ý nghĩa thống kê: paired t-test/Wilcoxon, $p < 0.05$.
- Ablation study 4 biến thể để cô lập đóng góp riêng của từng thành phần kiến trúc, theo tinh thần thiết kế thực nghiệm định lượng cho hệ thống RAG [1]:
    1. Full pipeline
    2. Bỏ Cấp 3
    3. Giữ cấp 3 nhưng thay AST-based bằng fixed-sized chunking
    4. Bỏ Progressive Disclosure

Dự kiến kết quả: Bộ số liệu benchmark (kèm kiểm định thống kê và ablation study) chứng minh tính ưu việt của chiến lược phân đoạn đề xuất so với cách tiếp cận truyền thống.

Hướng phát triển trong tương lai: Các giải pháp kỹ thuật đảm bảo chất lượng dữ liệu GitHub

Nội dung: Nếu điều kiện thời gian cho phép hoặc trong giai đoạn mở rộng đề tài, nhóm dự kiến bổ sung các giải pháp kỹ thuật nâng cao chất lượng dữ liệu đầu vào (Engineering – phục vụ tối ưu sản phẩm, không thuộc phạm vi cam kết khoa học chính thức của đề tài).

Phương pháp thực hiện: Hệ thống áp dụng bốn kỹ thuật kết hợp:

1. **Trọng số đóng góp thực tế:** tính dựa trên số dòng code thêm/xóa lấy từ GitHub API, thay vì đếm số lượng commit; tham khảo hướng khai thác dữ liệu hoạt động GitHub làm minh chứng năng lực ứng viên trong tuyển dụng kỹ thuật [4].
2. **Lọc theo tần suất commit:** loại bỏ các đóng góp dưới 10 lần commit.

3. **Chấm điểm độ trưởng thành repository:** dựa trên metadata (tuổi dự án, hoạt động cập nhật gần nhất, số star/fork, issue đang mở).
4. **Phát hiện sao chép mã nguồn (Code Similarity Detection):** tận dụng lại vector embedding đã có từ Nội dung 1–2 để so sánh cosine similarity theo hai phạm vi:
    - So khớp chéo giữa các ứng viên trong cùng một đợt tuyển (tương tự nguyên lý MOSS).
    - So khớp với tập tham chiếu boilerplate phổ biến (Spring Initializr, create-react-app...).

    Các cặp vượt ngưỡng tương đồng (dự kiến 0.85–0.9) được gắn cờ cảnh báo để HR/sinh viên xem xét thêm thay vì tự động loại bỏ.

Ngoài ra, một thuật toán regex cũng được dùng để loại bỏ tài khoản bot tự động commit.

Dự kiến kết quả: Hệ thống phát hiện và loại bỏ được các đóng góp thấp hoặc không thực chất, đồng thời gắn cờ cảnh báo các trường hợp nghi ngờ sao chép mã nguồn, giúp đảm bảo chất lượng minh chứng năng lực đưa vào hai phân hệ Doanh nghiệp và Sinh viên.

B2.3 Kế hoạch nghiên cứu

Giai đoạn 1 – chuẩn bị & thiết kế (tháng 1-2)

Tuần	Công việc chi tiết	Kết quả / Checkpoint
1-2	Khảo sát tài liệu liên quan (RAG, AST chunking, RAGAS); xác định rõ RQ1-RQ3 và metric tương ứng; thiết kế kiến trúc tổng thể hệ thống 2 phân hệ	Sơ đồ kiến trúc hệ thống + bảng mapping RQ → metric
3-4	Thiết kế schema database; lựa chọn & chốt bộ công cụ nền tảng (bge-m3, pgvector, Gemini 1.5 Flash, Tree-sitter)	Tài liệu kiến trúc kỹ thuật (Tech Spec) v1
	Soạn thảo Annotation Guideline (thang 3 mức 0/1/2); thu thập tập **pilot** 10 JD + 15 repo	Annotation Guideline v1
5-6	Cài đặt khung pipeline chunking rỗng (skeleton) cho cả AST-based và fixed-size, chưa cần tối ưu	Repo code khởi tạo, chạy được end-to-end trên 1 mẫu
	2 người gán nhãn độc lập trên tập pilot; tính Cohen's Kappa sơ bộ	**Checkpoint 1:** Kappa pilot ≥ 0.7 → nếu không đạt, họp hiệu chỉnh guideline ngay

		trong tuần 7–8 trước khi mở rộng
7-8	Hiệu chỉnh guideline (nếu cần); thiết kế chi tiết luồng fallback Tree-sitter → Line-based	Annotation Guideline v2 (final); đặc tả kỹ thuật fallback

Kết quả bàn giao Giai đoạn 1: Kiến trúc hệ thống hoàn chỉnh trên giấy; guideline gán nhãn đã kiểm chứng bằng pilot (Kappa >= 0.7); pipeline chunking chạy được ở mức khung sườn.

Giai đoạn 2 – Xây dựng & Thu thập mở rộng (Tháng 3-4)

Tuần	Công việc chi tiết	Kết quả / Checkpoint
9-10	Xây dựng backend + tích hợp GitHub API (OAuth, lấy repo/README/commit)	Module thu thập dữ liệu GitHub hoạt động
	Thu thập phần còn lại của dataset: đủ 50 JD (ITviec/TopCV) + 100 repo	Dataset thô đầy đủ 50 JD / 100 repo
11-12	Cài đặt hoàn chỉnh pipeline AST-based chunking (Java, JS/TS) + fallback Line-based	Pipeline AST chunking chạy ổn định trên toàn bộ repo Java/JS-TS trong dataset
	Gán nhãn ground truth đợt 1 (song song, không dồn cuối) — mục tiêu ~40% dataset	**Checkpoint 2:** ≥ 40% ground truth hoàn tất giữa tháng 4
13-14	Cài đặt pipeline fixed-size chunking (RecursiveCharacterTextSplitter, 500 token/overlap 50)	Pipeline fixed-size chunking hoàn chỉnh
	Triển khai hạ tầng Ollama local (Llama-3/Qwen2.5 + BGE-M3) cho các vòng thử nghiệm nội bộ, tiết kiệm chi phí API	Môi trường local sẵn sàng chạy thử ablation không tốn API
15-16	Hoàn thiện pipeline RAG cơ bản (retrieval + ranking) nối 2 chiến lược chunking vào chung 1 luồng đánh giá	Pipeline RAG end-to-end, sẵn sàng đo benchmark
	Gán nhãn ground truth đợt 2 — mục tiêu 100% dataset	**Checkpoint 3:** 100% ground truth hoàn tất trước đầu tháng 5

Kết quả bàn giao Giai đoạn 2: Dataset đầy đủ và đã gán nhãn 100%; 2 pipeline chunking hoàn chỉnh chạy được trên toàn bộ dữ liệu; hạ tầng local (Ollama) sẵn sàng phục vụ ablation không phát sinh chi phí API.

Giai đoạn 3 – Benchmark & Đo lường (Tháng 5)

Tuần	Công việc chi tiết	Kết quả / Checkpoint
17-18	Xây giao diện MVP 2 phân hệ (Doanh nghiệp / Sinh viên) — tối giản, tập trung luồng nghiệp vụ	Giao diện MVP hoạt động, kết nối được với pipeline backend
	Chạy thử nghiệm benchmark sơ bộ trên local (Ollama) để rà lỗi đo lường trước khi tốn chi phí API thật	Log kết quả sơ bộ, xác nhận pipeline đo Precision/Recall/F1/NDCG/MRR chạy đúng
19-20	Hoàn thiện module Skill Gap	Module Skill Gap hoạt động
	Chạy benchmark chính thức bằng Gemini 1.5 Flash (API thương mại) trên toàn bộ dataset cho cả 2 chiến lược chunking	Bộ số liệu benchmark chính thức đầy đủ (RQ1, RQ2, RQ3)

Kết quả bàn giao Giai đoạn 3: Hệ thống MVP hoàn chỉnh; bộ số liệu benchmark chính thức trả lời trực tiếp RQ1–RQ3.

Giai đoạn 4 – Phân tích & Báo cáo (Tháng 6)

Tuần	Công việc chi tiết	Kết quả / Checkpoint
21-22	Chạy ablation study 4 biến thể; kiểm định thống kê (paired t-test/Wilcoxon, $p < 0.05$)	Bảng kết quả ablation + kiểm định thống kê
	Tính chỉ số Ragas (Faithfulness, Context Precision) trên tập kết quả retrieval	Bảng chỉ số Ragas
23-24	Kiểm thử toàn hệ thống end-to-end; tổng hợp kết quả	Danh sách lỗi đã fix; kết quả tổng hợp cuối cùng
	Viết báo cáo tổng kết; chuẩn bị slide bảo vệ	Báo cáo tổng kết hoàn chỉnh + slide bảo vệ

Kết quả bàn giao Giai đoạn 4: Toàn bộ kết quả thực nghiệm (benchmark + ablation + kiểm định thống kê) và báo cáo tổng kết sẵn sàng nộp/bảo vệ.

B3. Kết quả dự kiến

Tổng hợp từ 4 giai đoạn thực hiện ở mục B2.3, đề tài dự kiến đạt được các kết quả sau:

- **Bằng chứng thực nghiệm định lượng** trả lời trực tiếp ba câu hỏi nghiên cứu (Precision/Recall/F1@k cho RQ1, NDCG/MRR cho RQ2, token/latency cho RQ3), kèm kiểm định ý nghĩa thống kê và ablation study, cho thấy ảnh hưởng thực sự của chiến lược phân đoạn mã nguồn đến chất lượng truy xuất, xếp hạng, và chi phí tính toán – đây là đóng góp khoa học cốt lõi của đề tài.
- **Hệ thống demo (MVP)** minh họa luồng đối sánh JD – hồ sơ GitHub dựa trên bằng chứng năng lực: đăng JD → bóc tách kỹ năng → đối sánh hồ sơ GitHub → xếp hạng ứng viên → báo cáo Skill Gap.
- **Ở mức mở rộng (Hướng phát triển):** một số giải pháp kỹ thuật bổ trợ (phát hiện sao chép mã nguồn chéo giữa các ứng viên, phát hiện bot, đánh giá độ trưởng thành repository) có thể được tích hợp thêm để nâng cao chất lượng dữ liệu đầu vào, tuy không thuộc phạm vi cam kết bắt buộc của đề tài.
- **Cơ sở để phát triển:** thành sản phẩm hoàn chỉnh hoặc mở rộng nghiên cứu ở quy mô lớn hơn, ví dụ tích hợp thêm các nguồn minh chứng khác ngoài GitHub (LinkedIn, chứng chỉ trực tuyến, bài viết kỹ thuật).

B4. Tài liệu tham khảo

[1]. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks; Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela; 34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada, 2020.

[2]. cAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Tree; Yilin Zhang, Xinran Zhao, Zora Zhiruo Wang, Chenyang Yang, Jiayi Wei, Tongshuang Wu; Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 8106–8116, 2025.

[3]. RAGAS: Automated Evaluation of Retrieval Augmented Generation; Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert; Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, pp. 150–158, 2024.

[4]. Improving Evidence-Based Tech Hiring with GitHub-Supported Resume Matching; Swanand Vaishampayan, Muhammad Ali Gulzar, Chris Brown; IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2024.

[5]. CodeCV: Mining Expertise of GitHub Users from Coding Activities; Daniel Atzberger, Nico Scordialo, Tim Cech, Willy Scheibel, Matthias Trapp, Jürgen Döllner; Proceedings of the 2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM), pp. 143–147, 2022.

Ngày 29 tháng 8 năm 2026	*Ngày 29 tháng 8 năm 2026*
Giảng viên hướng dẫn	**Chủ nhiệm đề tài**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên)
Ths.Trần Thị Hồng Yến	Phạm Viết Quang Huy